Page No.:
Date:
YOUVA

Assignment - 3

Problem Statement: Write a C program to create a student database using array of structure. Apply searching and sorting techniques

Objective:
1) To understand use of an array of structures for maintaining records
2) To implement and analyze searching techniques
3) To implement and analyze sorting techniques.
4) To compose insertion Sort, Selection Sort, Shell Sort.

Theory:
Searching and Sorting are two of the most fundamental operations in data structures and algorithms. They are widely used in computer application such as database management systems, search engines, operating systems, and information retrieval systems. Efficient recycling and sorting techniques improves the performance of software by reducing the time required to locate and organize data

Searching
The process of finding the location of a specific element, called key within a collection of data. If the element is present, the search operation its position; otherwise, it indicates that the element is not found. Searching is an essential operation in applications such as student record management, library systems, banking sea systems, inventory management and online shopping.

Types of Searching

Linear Search

Linear search also known as Sequential Search is the simplest searching technique. In this method each element of the list is compared one by one with the target element until a match is found or the entire list has been searched.

Binary Search

Binary Search is an efficient algorithm that works only on sorted data. It repeatedly divides the search interval into two halves until the target element is found or the search space becomes empty.

Sorting

Sorting is the process of arranging data elements in a specific order usually ascending or descending. Sorted data allows faster searching easier analysis and efficient data processing.

Types of Sorting

Bubble Sort

Bubble Sort repeadedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to its current position, similar to bubbles rising to the surface.

Selection Sort

Selection Sort repetedly selects the smallest element from the unsorted portion of the list and places it at the begining.

Insertion Sort

Insertion Sort builds a sorted list one element at a time by inserting each new element into its correct position in the already sorted portion of the array.

Shell Sort

Shell Sort allows the exchange of elements that are far apart. It sorts elements by comparing and swapping items seperated by a specific gap, gradually reducing the gap until it becomes 1.

Merge Sort

Merge Sort is a divide and conquer algorithm that divides the array into smaller subarrays, sorts each subarray recursively, and then merges them into a sorted array.

Quick Sort

Quick Sort is another divide and conquer algorithm. It selects a pivot element and partitions the array into two parts such that elements smaller than the pivot are placed before it and larger elements after it. The process is repeated recursively.

Implementation

- Platform
  - 64 bit Open source Linux or its derivative
  - Open Source C Programming tool like gcc/ Eclipse Editor.
- Test Conditions :-
  - Input at least five records
  - Search a student in the record as per the roll number (Linear search)
  - Arrange a list of students roll call list wise (Use Insertion Sort Selection sort, Shell sort)
  - Search a student as per roll number in the sorted list (Binary search)
  - Test the searching and sorting methods for at least five

- Pseudo Code

1) Linear Serch

Linear Serch(arr, n, target_roll)
Input: Arry of Student structure arr of size n, target_roll
Output: Index of target_roll if found, else -1
1) For i = 0 to n-1 do:
   a) If arr[i].roll_no == target_roll then:
      i. Return i
2) Return -1

2) Binary Code Serch

Algorithm Binary Search (arr, n, target_roll)
Input: Sorted array of student students arr of size n, target roll
Output: Index of target_roll if found, else -1
1. Set low = 0, high = n-1
2. While low <= high do:
   a. Set mid = low + (high - low) / 2
   b. If arr[mid].rollno == target roll then:
      i. Return mid
   c. Else if arr[mid].roll no < target_roll then:
      i. set low = mid+1
   d. Else:
      i. Set high = mid - 1
3. Return -1

3) Selection Sort

Algorithm SelectionSort(arr, n)
Input: Array of student structures arr of size n
Output: Array sorted by roll_no in ascending order.
1. For i = 0 to n-2, do:
   a. Set min_idx = i

b. For j = i+1 to n-1 do:
   i. If arr[j].roll_no < arr[min_idx].roll_no then:
      1. Set min_idx = j
c. If min_idx != i then:
   i. Swap arr[i] and arr[min_idx]

4) Insertion Sort
Algorithm InsertionSort(arr, n)
Input: Array of student structures arr of size n
Output: Array sorted by roll-no in ascending order
1. For i = 1 to n-1 do:
   a. Set key = arr[i]
   b. Set j = i-1
   c. While j >= 0 and arr[j].roll_no > key.roll_no do:
      i. Set arr[j+1] = arr[j]
      ii. Set j = j-1
   d. Set arr[j+1] = key.

5) Shell Sort
Algorithm ShellSort(arr, n)
Input: Array of student structures arr of size n
Output: Array sorted by roll-no in ascending order
1. For gap = n/2 down to 1 (divide gap by 2 each step) do:
   a. For i = gap to n-1 do:
      i. Set temp = arr[i]
      ii. Set j = i
      iii. While j >= gap and arr[j-gap].rollno > temp.rollno
         1. Set arr[j] = arr[j-gap]
         2. Set j = j-gap
      iv. Set arr[j] = temp

M T W T F S S
Page No.:
Date:
YOUVA

Time Complexity

Operation	Best Case Complexity	Average Case
Linear Search	O(1)	
Binary Search	O(1)	
Selection Sort	O(n²)	
Insertion Sort	O(n)	
Shell Sort	O(n log n)	

FAQs

1) What is the meaning of database? How to maintain in C?

Ans. A database is an organized collection of structured data for easy access, management, and updating. In C, it is maintained in RAM using an array of structures or permanently on disk using file operations

2) What are the applications of searching?

Ans.
- Record Retrieval
- E-Commerce
- Operating Systems
- Database Systems

3 Compare linear search and binary search?

Parameter	Linear Search	Binary Search
Data Requirement	Works on unsorted and sorted data	Requires pre-sorted data
Time Complexity	O(n)	O(log n)
Access Method	Sequential	Random / Direct
Efficiency	Best for small datasets	Best for large datasets

4. What are the applications of sorting?
- Enabling Fast Search
- Data Presentation
- Data Processing
- Task Prioritization

5. Compare and contrast selection, insertion sort and shell sort?
- Selection Sort: Finds the minimum element in each pass and swaps it to the front. Always takes $O(n^2)$ comparisions regardless of initial order.
- Insertion Sort: Inserts each element into its sorted position one by one. Highly efficient $O(n)$ for nearly sorted data but $O(n^2)$ in the worst case.
- Shell Sort: An optimized insertion sort that swaps elements far apart using decreasing gaps, breaking long distance inversions early ($O(n^{1.3})$ average time).

6. What is external & internal sorting, In place, Stable Features of Sorting Algorithms?
- Internal Sorting: Sorts data that fits completly within main memory (RAM).
- External Sorting: Sorts massive data sets that exceeds RAM capacity using secondary storage (disks).
- In place Sorting: Uses a constant amount of extra space ($O(1)$ auxillary memory).
- Stable Sorting: Preserves the relative input order of duplicate elements after sorting.

```
#include <stdio.h>
```

```c
#include <string.h>

#define MAX 50

// Structure to store student details
struct Student {
    int roll_no;
    char name[50];
    float sgpa;
};

// Function prototypes
void acceptRecords(struct Student s[], int n);
void displayRecords(struct Student s[], int n);
void linearSearch(struct Student s[], int n, int key);
void binarySearch(struct Student s[], int n, int key);
void insertionSort(struct Student s[], int n);
void selectionSort(struct Student s[], int n);
void shellSort(struct Student s[], int n);

int main() {
    struct Student s[MAX], temp[MAX];
    int n = 0, choice, key, i;

    printf("=== Student Database System ===\n");
    printf("Enter number of student records (minimum 5): ");
    scanf("%d", &n);

    if (n < 5) {
```

```
    printf("Note: Program requires at least 5 records as per lab specifications.\n");
}

acceptRecords(s, n);

do {
    printf("\n-------------------------------------\n");
    printf("MENU:\n");
    printf("1. Display Records\n");
    printf("2. Linear Search (by Roll No)\n");
    printf("3. Sort by Roll No (Insertion Sort)\n");
    printf("4. Sort by Roll No (Selection Sort)\n");
    printf("5. Sort by Roll No (Shell Sort)\n");
    printf("6. Binary Search (by Roll No - Requires Sorted List)\n");
    printf("7. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            displayRecords(s, n);
            break;

        case 2:
            printf("Enter Roll Number to search (Linear Search): ");
            scanf("%d", &key);
            linearSearch(s, n, key);
            break;
```

```
case 3:
    // Copy to temp array to preserve original order
    for (i = 0; i < n; i++) temp[i] = s[i];
    insertionSort(temp, n);
    printf("\n--- Database Sorted using Insertion Sort ---\n");
    displayRecords(temp, n);
    break;

case 4:
    for (i = 0; i < n; i++) temp[i] = s[i];
    selectionSort(temp, n);
    printf("\n--- Database Sorted using Selection Sort ---\n");
    displayRecords(temp, n);
    break;

case 5:
    for (i = 0; i < n; i++) temp[i] = s[i];
    shellSort(temp, n);
    printf("\n--- Database Sorted using Shell Sort ---\n");
    displayRecords(temp, n);
    break;

case 6:
    // Binary search requires the primary dataset to be sorted first
    shellSort(s, n);
    printf("\n[Info: Database automatically sorted before Binary Search]\n");
    printf("Enter Roll Number to search (Binary Search): ");
    scanf("%d", &key);
    binarySearch(s, n, key);
```

```
            break;

        case 7:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 7);

    return 0;
}

// Function to accept student records
void acceptRecords(struct Student s[], int n) {
    for (int i = 0; i < n; i++) {
        printf("\nEnter details for Student %d:\n", i + 1);
        printf("Roll Number: ");
        scanf("%d", &s[i].roll_no);
        printf("Name: ");
        scanf("%s", s[i].name);
        printf("SGPA: ");
        scanf("%f", &s[i].sgpa);
    }
}

// Function to display student records
void displayRecords(struct Student s[], int n) {
```

```
    printf("\n%-10s %-20s %-10s\n", "Roll No", "Name", "SGPA");
    printf("----------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-10d %-20s %-10.2f\n", s[i].roll_no, s[i].name, s[i].sgpa);
    }
}

// Linear Search implementation
void linearSearch(struct Student s[], int n, int key) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (s[i].roll_no == key) {
            printf("\nStudent Found at index %d (Position %d):\n", i, i + 1);
            printf("Roll No: %d | Name: %s | SGPA: %.2f\n", s[i].roll_no, s[i].name,
s[i].sgpa);
            found = 1;
            break;
        }
    }
    if (!found) {
        printf("\nStudent with Roll No %d not found.\n", key);
    }
}

// Binary Search implementation
void binarySearch(struct Student s[], int n, int key) {
    int low = 0, high = n - 1, mid, found = 0;

    while (low <= high) {
```

```
        mid = low + (high - low) / 2;

        if (s[mid].roll_no == key) {
            printf("\nStudent Found at index %d (Position %d):\n", mid, mid + 1);
            printf("Roll No: %d | Name: %s | SGPA: %.2f\n", s[mid].roll_no, s[mid].name,
s[mid].sgpa);
            found = 1;
            break;
        }
        if (s[mid].roll_no < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    if (!found) {
        printf("\nStudent with Roll No %d not found.\n", key);
    }
}

// Insertion Sort implementation
void insertionSort(struct Student s[], int n) {
    int i, j;
    struct Student key;
    for (i = 1; i < n; i++) {
        key = s[i];
        j = i - 1;
        while (j >= 0 && s[j].roll_no > key.roll_no) {
            s[j + 1] = s[j];
```

```
            j = j - 1;
        }
        s[j + 1] = key;
    }
}

// Selection Sort implementation
void selectionSort(struct Student s[], int n) {
    int i, j, min_idx;
    struct Student temp;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (s[j].roll_no < s[min_idx].roll_no) {
                min_idx = j;
            }
        }
        if (min_idx != i) {
            temp = s[i];
            s[i] = s[min_idx];
            s[min_idx] = temp;
        }
    }
}

// Shell Sort implementation
void shellSort(struct Student s[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
```

```
            struct Student temp = s[i];
            int j;
            for (j = i; j >= gap && s[j - gap].roll_no > temp.roll_no; j -= gap) {
                s[j] = s[j - gap];
            }
            s[j] = temp;
        }
    }
}
=== Student Database System ===
Enter number of student records (minimum 5): 5

Enter details for Student 1:
Roll Number: 104
Name: Alice
SGPA: 8.5

Enter details for Student 2:
Roll Number: 101
Name: Bob
SGPA: 9.1

Enter details for Student 3:
Roll Number: 105
Name: Charlie
SGPA: 7.8

Enter details for Student 4:
Roll Number: 102
```

Name: David

SGPA: 8.9

Enter details for Student 5:

Roll Number: 103

Name: Eva

SGPA: 9.4

MENU:

1. Display Records

2. Linear Search (by Roll No)

3. Sort by Roll No (Insertion Sort)

4. Sort by Roll No (Selection Sort)

5. Sort by Roll No (Shell Sort)

6. Binary Search (by Roll No - Requires Sorted List)

7. Exit

Enter your choice: 1

Roll No	Name	SGPA
104	Alice	8.50
101	Bob	9.10
105	Charlie	7.80
102	David	8.90
103	Eva	9.40

MENU:

1. Display Records

2. Linear Search (by Roll No)

3. Sort by Roll No (Insertion Sort)

4. Sort by Roll No (Selection Sort)

5. Sort by Roll No (Shell Sort)

6. Binary Search (by Roll No - Requires Sorted List)

7. Exit

Enter your choice: 2

Enter Roll Number to search (Linear Search): 102

Student Found at index 3 (Position 4):

Roll No: 102 | Name: David | SGPA: 8.90

MENU:

1. Display Records

2. Linear Search (by Roll No)

3. Sort by Roll No (Insertion Sort)

4. Sort by Roll No (Selection Sort)

5. Sort by Roll No (Shell Sort)

6. Binary Search (by Roll No - Requires Sorted List)

7. Exit

Enter your choice: 3

--- Database Sorted using Insertion Sort ---

Roll No Name SGPA

---

101 Bob 9.10

102 David 8.90

103 Eva 9.40

104 Alice 8.50

105 Charlie 7.80

MENU:

1. Display Records

2. Linear Search (by Roll No)

3. Sort by Roll No (Insertion Sort)

4. Sort by Roll No (Selection Sort)

5. Sort by Roll No (Shell Sort)

6. Binary Search (by Roll No - Requires Sorted List)

7. Exit

Enter your choice: 6

[Info: Database automatically sorted before Binary Search]

Enter Roll Number to search (Binary Search): 105

Student Found at index 4 (Position 5):

Roll No: 105 | Name: Charlie | SGPA: 7.80

MENU:

1. Display Records

2. Linear Search (by Roll No)

3. Sort by Roll No (Insertion Sort)

4. Sort by Roll No (Selection Sort)

5. Sort by Roll No (Shell Sort)

6. Binary Search (by Roll No - Requires Sorted List)

7. Exit

Enter your choice: 7

Exiting program...